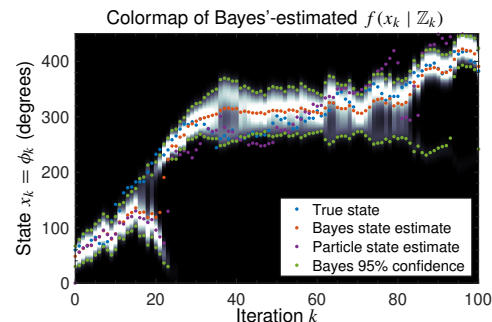
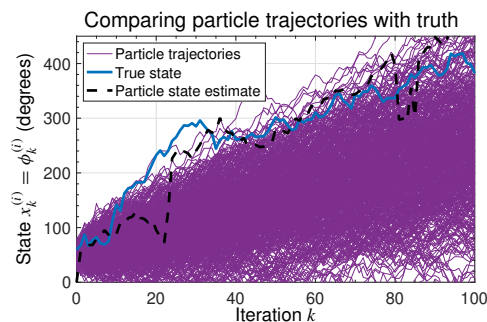


Review: Output from basic SIS method



- The particles, with true state trajectory overlaid, are shown.
- Particles represent many possible trajectories to choose from.
- Low particle density near true state; state estimate is similar to Bayes' optimal.
- Need to dig deeper to see if anything is going wrong with our estimator.



Code to estimate pdf from particles



```
% Convert particles to pdf estimate using bump function
nR      = 500; % Evaluate pdf over this many "bins"
xGrid   = linspace(0,450/(180/pi),nR).'; % x locs of bins
estPDF   = zeros(nR,nIter); % estimate of f(x{k}|Z{k})
xLowBnd  = zeros(nIter,1); xHiBnd = xLowBnd; % Confidence-interval bounds
kernelFN = @(x) (1-x.^2).^2.*(abs(x)<=1); % Kernel "bump" function to use

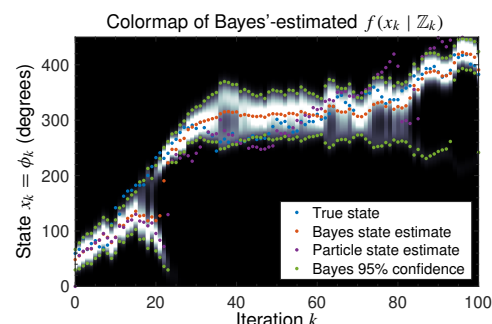
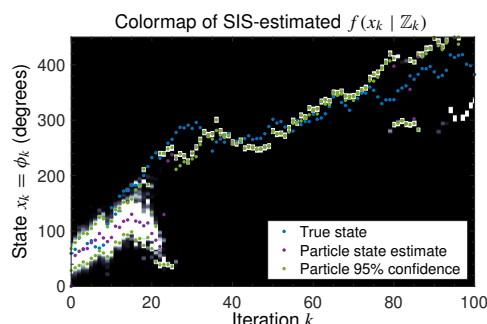
for k=1:nIter % repeat for every time iteration
    [xps,iSort] = sort(xp(:,k)); % Sort particles by increasing value
    wps         = wp(iSort,k); % Weights must agree with particles
    wSum        = cumsum(wps); % Cumulative sum of sorted weights
    xLowBnd(k)  = xps(1); % Compute lower confidence "x"
    if wSum(1) < 0.025, xLowBnd(k) = xps(find(wSum<0.025,1,'last')); end
    xHiBnd(k)   = xps(find(wSum>0.975,1,'first')); % Upper confidence "x"

    for kp = 1:nPart % Add bumps together for all particles; kernel width = 0.1
        estPDF(:,k) = estPDF(:,k) + wps(kp)*kernelFN((xGrid-xps(kp))/0.1);
    end
    estPDF(:,k) = estPDF(:,k)./(sum(estPDF(:,k))*mean(diff(xGrid))); % Normalize
end
```

Digging deeper: Comparing pdfs



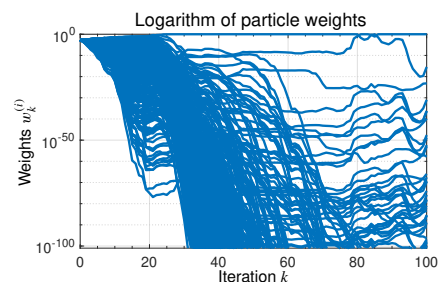
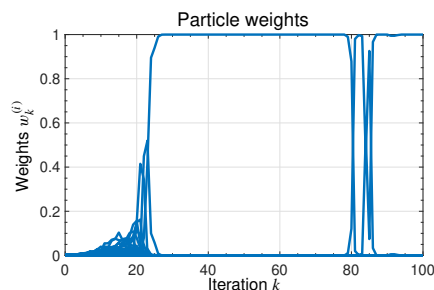
- Comparing SIS-estimated marginal pdf using kernel functions to the Bayes'-estimated pdf, we get the hint of a problem.
- The SIS estimate starts out okay, but quickly collapses to a single likely trajectory.
- Probably not what we want.



Digging deeper: Examining the particle weights



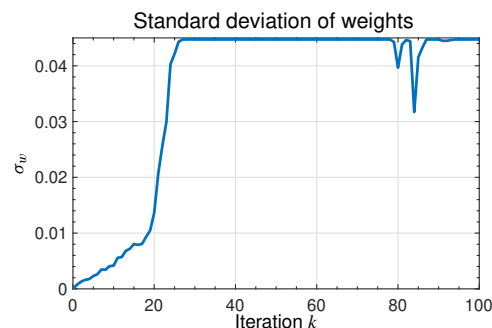
- We look at particle weights (on natural and log scales), which all start at the same value, $1/N_p = 1/500$.
 - But very quickly we discover that one particle “takes over.”
 - The other particles’ weights decay to values close to zero; so, those particles have no influence on the results. A lot of computation that produces no benefit.



Digging deeper: Examining the particle weights



- We might also look at the standard deviation of the weights.
- The increasing value shows that weights are becoming very unequal, and some particles are being ignored.
- This is undesirable—we would prefer for all particles to participate more-or-less equally in the computation of the estimated pdf.
- Otherwise, we are wasting a lot of CPU, and are likely giving up accuracy because of all the improbable particles that participate in the average.



Summary



- In this lesson, you learned about a vulnerability of the basic SIS algorithm: particle trajectories can become irrelevant.
 - The SIS estimate quickly collapses to a dependence on a single likely trajectory.
- In this example, the estimated state was still reasonably close to the Bayes'-optimal solution, but this is not true in general.
 - Even so, the confidence bounds were far from the Bayes'-optimal solution.
- Examining the particle weights, we conceive of a possible diagnostic: Are the weights all similar to one another, or does one weight take over?
- Similarly, does the standard deviation of the weights stay low, or does it increase to some bound?
- In the next lesson, we will use these diagnostics to propose an improvement to the basic SIS algorithm.